

Exploitation — versionnage et retour arrière

Ce document est fait pour être lu à 3 h du matin. Les commandes sont copiables telles quelles ; les justifications sont en retrait.

1. Qu'est-ce qu'une version ?

Une version de Mardik **n'est pas un commit** et **n'est pas un fichier de poids**. C'est le bundle `models/vX/config.yaml` : tout ce qui change la sortie du système à code constant.

Champ	v1	v2	Effet d'un changement
strategie	monolithique	map_reduce_clauses	change le moteur appelé par la gateway
prompt	contrat entier	une section	change la sortie sur tout le jeu d'éval
schema_sortie	null (texte)	JSON clauses/extrait/confiance	change le contrat de données modèle → pipeline
parametres.temperature	0.2	0.0	change la reproductibilité de la note
parametres.contexte_max_caracteres	16 000 pour tout le contrat	6 000 par section	c'est la troncature de la v1
parametres.concurrence	—	8	change la latence et la charge fournisseur
modele	\${LLM_MODEL}	\${LLM_MODEL}	change tout, y compris le coût

Le code ne connaît aucune de ces valeurs : il lit le bundle. C'est ce qui rend un rollback instantané — revenir en arrière, c'est servir un autre fichier, pas redéployer une application.

Changer seulement le prompt suffit à créer une nouvelle version : la sortie change, donc la note du gate change, donc l'artefact doit porter une étiquette distincte. **Changer seulement le modèle de base** aussi — et c'est le cas le plus traître, parce que rien dans le dépôt ne bouge : c'est pourquoi `modele` est recopié dans le manifeste au moment de l'étiquetage.

2. Schéma d'étiquetage

Étiquettes `vMAJEUR.MINEUR.CORRECTIF`, sur la **sortie**, jamais sur le code :

- MAJEUR** — le contrat d'API change (nouveau champ obligatoire, champ retiré, sémantique d'un champ modifiée). Un client doit être adapté. La v1 reste servie en parallèle : c'est précisément le passage v1 → v2.
- MINEUR** — la sortie change sans casser le contrat : nouveau prompt, nouvelle stratégie, nouveau modèle de base, nouveaux paramètres. Le client ne change pas, mais les résultats oui — donc gate obligatoire.
- CORRECTIF** — rien ne change pour le client ni pour la sortie attendue : correction de parsing, de journalisation, de concurrence.

Chaque étiquette dépose dans `ops/registry/<version>/` :

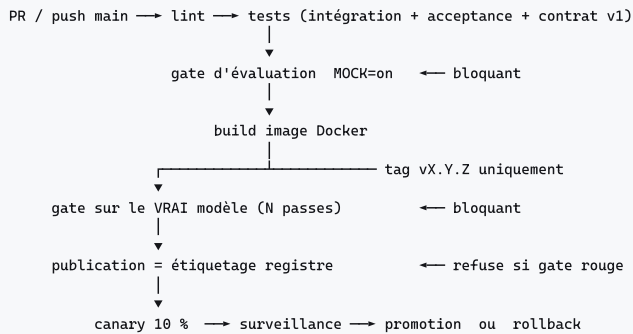
config.yaml	le bundle tel quel, figé — une version étiquetée est immuable
manifest.json	version, commit, empreinte (SHA-256 du bundle), modele, strategie, note_eval, seuil, essais_eval, dispersion_note, latence_p95_ms, cout_moyen_eur, date

Retrouver ce qui tourne en production :

```
curl -s localhost:8000/gateway/etat | jq # active, canary, pourcentage
cat ops/registry/$(curl -s localhost:8000/gateway/etat | jq -r .active)/manifest.json
```

L'empreinte est le hash du `config.yaml`, pas du commit. Deux commits différents qui laissent le bundle identique produisent la même empreinte : c'est voulu, le modèle n'a pas changé.

3. Chaîne de livraison



Ce que la CI vérifie sans le vrai modèle (MOCK=on, à chaque fusion) : la mécanique. Le découpage ne perd pas de texte, une sortie hors schéma est écartée sans casser l'analyse, la consolidation ne laisse pas de doublon, la publication est refusée quand le gate échoue, le rollback tient en une opération, le client v1 répond comme avant. Tout cela est déterministe, gratuit et rapide : c'est ce qui a le droit de bloquer une PR.

Ce qui exige le vrai modèle (sur tag ou `workflow_dispatch`) : la note. Les fixtures rejouent des réponses enregistrées un jour donné ; elles ne disent rien de la qualité du modèle d'aujourd'hui. Un gate MOCK vert ne prouve pas qu'une v2 est meilleure — il prouve qu'elle n'est pas cassée.

Ce qui bloque, concrètement : `ops.deploy publier` rejoue le gate et lève `ErreurDéploiement` si la note est sous le seuil, si un contrat passe sous son `seuil_note`, si la P95 atteint 8 s ou si le coût moyen atteint 0,15 €. Il n'y a pas de chemin vers le registre qui contourne cette vérification — ni par la CI, ni par la console.

4. Déploiement progressif

```
python -m ops.deploy canary v2.0.0 --pourcentage 10
python -m ops.deploy surveiller --boucle --intervalle 30
```

Palier	Durée minimale	Condition pour monter
10 %	2 h ou ≥ 200 analyses	aucune dérive, confiance $\geq 0,75$
30 %	1 jour ouvré	idem + aucun retour juriste négatif
100 % (promouvoir)	—	décision humaine

Qui décide quoi. La *descente* est automatique : `surveiller` déclenche le rollback seul, parce qu'attendre un humain pendant que la production dérive coûte des analyses fausses. La *montée* reste manuelle : promouvoir, c'est accepter un risque, et aucun des signaux disponibles ne mesure ce qui compte vraiment pour le client — la justesse juridique des clauses relevées. Une boucle qui promeut toute seule sur des indicateurs techniques promeut un modèle rapide et pas cher, pas un modèle juste.

La gateway relit `index.json` à chaque requête : canary, promotion et rollback prennent effet immédiatement, sans redémarrage ni coupure.

5. Procédure de rollback

Une commande, toujours la même :

```
python -m ops.deploy rollback --motif "ce qui vous a alerté"
```

Elle gère les deux cas sans que vous ayez à les distinguer : s'il y a un canary, elle le retire ; sinon elle rend la main à précédente.

Qui a le droit : toute personne d'astreinte. Aucune validation préalable — un rollback est réversible (canary ou promouvoir le rejouent), l'incident qu'il évite ne l'est pas.

Vérifier que ça a marché, dans l'ordre :

```
curl -s localhost:8000/gateway/etat | jq          # active = version attendue, canary = null
python scripts/client_v1.py                      # le client historique répond
python -m ops.dashboard --fenetre 300           # le trafic est bien retombé sur l'active
tail -1 ops/registry/journal.jsonl | jq         # l'entrée de rollback est écrite
```

Ce qui est inscrit au journal (`ops/registry/journal.jsonl`, une ligne JSON par événement) : `date`, `evenement` : "rollback", `motif` (le signal, pas un commentaire), `avant` et `apres` avec `active` / `canary` / `canary_percent`. C'est ce qui permet, le lendemain, de répondre à « qu'est-ce qui l'a déclenché, et quand ? » sans reconstituer l'histoire à partir des logs applicatifs.

6. Surveillance et seuils

```
python -m ops.deploy surveiller --boucle --fenetre 120
```

Signal	Seuil	Pourquoi cette valeur
confiance moyenne	< 0,70	le client relit en dessous de 0,7 ; une version qui y descend en moyenne ne lui fait plus gagner de temps
taux d'erreur	> 10 %	au-delà, un juriste sur dix retombe sur la saisie manuelle
latence P95	> 8 000 ms	contrainte contractuelle (docs/besoin_client.md)
fenêtre	120 s	assez court pour réagir, assez long pour lisser un pic fournisseur
minimum de mesures	10	le garde-fou le plus important
faible_confiance	0,70	en dessous, le cas part en capture pour le jeu d'éval (boucle 3)
minimum_promotion	30	une promotion engage plus qu'un rollback : on en exige plus
marge_promotion	0,05	tolérance entre la confiance de production et celle du gate

Ces seuils vivent dans `ops/seuils.json`, **pas dans le code** : les ajuster est une opération d'exploitation, pas une livraison. Et chaque ajustement est tracé :

```
python -m ops.deploy seuil faible_confiance 0.75 --auteur "eq-ia" --motif "P10 observé à 0,71 sur 7 jours"
```

Un seuil qu'on remonte discrètement parce que l'alerte sonne trop souvent est la façon la plus courante de désarmer une surveillance sans le dire. D'où la trace obligatoire, avec auteur et motif.

Sur minimum . Sans lui, trois requêtes malheureuses au redémarrage ou dans un creux de nuit suffisent à déclencher un rollback. Une boucle automatique qui se déclenche à tort est pire qu'une boucle absente : on finit par la désactiver, et on perd aussi les vraies détections.

Faux positifs connus et conduite à tenir :

- Lot de contrats atypiques* (contrats anglais, scans mal océsés) : la confiance baisse légitimement, le modèle n'est pas en cause. Le journal conserve le motif ; si la confiance remonte au lot suivant, ne pas re-promouvoir dans la foulée — vérifier d'abord sur l'évaluation enrichie.
- Dégradation du fournisseur* : latence et erreurs montent ensemble, la confiance ne bouge pas. Le rollback ne répare rien — la v1 appelle le même fournisseur. Vérifier `/health` et le proxy avant de conclure sur la version.
- Score composite bas sur extraits reformulés* : le score pénalise les citations non littérales (`pipeline/confiance.py`). Un changement de prompt qui pousse le modèle à résumer fait chuter la confiance sans que la qualité baisse. C'est un vrai signal — mais sur le prompt, pas sur la production.

Les seuils sont des **arguments de la commande**, pas des constantes : les ajuster se fait sans redéploiement, et l'ajustement part dans le journal avec le reste.

6 bis. Les deux autres boucles

Boucle 2 — promotion canary pilotée par les métriques

```
python -m ops.deploy promouvoir-auto --fenetre 600
```

Monte le canary d'un palier (10 → 30 → 100 %) si, et seulement si : la fenêtre compte au moins `minimum_promotion` mesures, aucune condition de rollback n'est remplie, et la confiance observée atteint celle mesurée **par le gate** à la sortie de la chaîne, à `marge_promotion` près.

Dérive par rapport à quoi ? Par rapport à `confiance_moyenne` figée dans le manifeste de la version. Une moyenne glissante de production absorberait lentement la dérive qu'elle est censée détecter.

Un refus est journalisé (`promotion_refusee`) au même titre qu'une promotion : savoir pourquoi la v2 **n'est pas** montée est aussi utile que l'inverse — c'est ce qui manque quand une migration stagne sans qu'on sache sur quoi.

Le passage à 100 % par `promouvoir` reste une décision humaine : aucun signal disponible ne mesure la justesse juridique des clauses relevées.

Boucle 3 — enrichissement du jeu d'évaluation

Automatique côté capture, humaine côté versement :

```
# 1. capture : automatique, sur toute analyse servie sous le seuil de confiance
# (branchée sur la gateway, PAS sur le moteur – sinon le gate capturerait
# ses propres contrats à chaque exécution)
# 2. relecture : page /pilotage, ou
curl -s localhost:8000/api/captures | jq '.attente[] | {id, confiance_observed, anonymisation}'
# 3. versement
curl -X POST localhost:8000/api/captures/<id>/valider -H 'content-type: application/json' -d '{"clauses_attendues": ["résiliation","durée"], "auteur": "
# 4. le gate le rejoue à la fusion suivante, sans configuration supplémentaire
make eval
```

Pourquoi une relecture humaine. Elle tient deux rôles. *Exactitude* : une confiance basse veut dire que le modèle doute, pas qu'il s'est trompé — verser sa sortie telle quelle comme vérité de référence graverait une erreur dans le jeu qui garde la chaîne. *Confidentialité* : la pseudonymisation automatique (eval/anonymisation.py) traite ce qui a une forme reconnaissable — IBAN, SIREN, montants, adresses, courriels, raisons sociales — mais laissera passer un nom écrit au fil du texte. Le relecteur est le dernier contrôle avant que le cas ne parte dans Git et dans l'intégration continue.

Le cas versé entre avec un seuil_note bas (0,5). Il a été capturé parce qu'il est difficile ; l'ajouter au seuil des contrats de référence ferait échouer le gate dès la fusion suivante et pousserait l'équipe à retirer le cas plutôt qu'à améliorer le modèle. Le jeu d'évaluation doit pouvoir accueillir ce qui fait mal.

7. Preuve d'exécution

Transcript d'un cycle complet (MOCK=on, registre isolé) :

```
$ python -m eval.run_eval --version v1
K0 c10 note=0.40 (seuil 0.8) manquantes: droit applicable, garantie, résiliation...
K0 c12 note=0.33 (seuil 0.8) manquantes: confidentialité, force majeure, résiliation...
note globale = 0.853 | P95 = 1 ms | coût moyen = 0.0067 € | dispersion = 0.000
GATE : ÉCHEC - contrats sous leur seuil : c07, c10, c12
```

La v1 est refusée par son propre gate sur les trois contrats longs : la douleur du client est devenue une mesure.

```
$ python -m eval.run_eval --version v2
OK c07 note=1.00 (seuil 0.8)
OK c10 note=1.00 (seuil 0.8)
OK c12 note=1.00 (seuil 0.8)
note globale = 1.000 | P95 = 21 ms | coût moyen = 0.0329 € | dispersion = 0.000
GATE : PASSE

$ python -m ops.deploy publier v2.0.0
{'version': 'v2.0.0', 'commit': '8c59599', 'empreinte': '5e770142a8a8',
 'strategie': 'map_reduce_clauses', 'note_eval': 1.0, 'seuil': 0.75,
 'latence_p95_ms': 19.5, 'cout_moyen_eur': 0.032912}

$ python -m ops.deploy canary v2.0.0 --pourcentage 20
{'active': 'v1.0.0', 'canary': 'v2.0.0', 'canary_percent': 20}

$ # 50 analyses sur la gateway
trafic servi : {'v1.0.0': 40, 'v2.0.0': 10} # exactement 20 %

$ python -m ops.dashboard --fenetre 600
version   trafic  req   P50 ms   P95 ms   err  confiance   coût €
v1.0.0    80.0%  40     0         1   0.0%    -         0.1335
v2.0.0    20.0%  10     7        10   0.0%    0.735    0.1928

$ python -m ops.deploy surveiller --fenetre 600
{'version': 'v2.0.0', 'mesures': 10, 'derive': False, 'rollback': False}

$ # dérive du score injectée sur le canary (équivalent de DRIFT=score)
$ python -m ops.deploy surveiller --fenetre 600
{'version': 'v2.0.0', 'mesures': 25, 'derive': True,
 'motif': 'score de confiance moyen 0.582 < 0.7', 'rollback': True}

$ tail -1 ops/registry/journal.jsonl
{"date": "2026-10-01T09:01:37+00:00", "evenement": "rollback",
 "motif": "score de confiance moyen 0.582 < 0.7",
 "avant": {"active": "v1.0.0", "canary": "v2.0.0", "canary_percent": 20},
 "apres": {"active": "v1.0.0", "canary": null, "canary_percent": 0}}
```

Les deux autres boucles, même session :

```
$ python -m ops.deploy promouvoir-auto --fenetre 900
{'version': 'v2.0.0', 'pourcentage': 20, 'mesures': 10, 'promue': False,
 'motif': 'fenêtre incomplète : 10/30 mesures'} # la boucle s'abstient

$ python -m ops.deploy promouvoir-auto --fenetre 900 --minimum 10
{'promue': True, 'palier': 30, 'motif': 'palier 20 % -> 30 %',
 'indicateurs': {'score_moyen': 0.7303, 'score_p10': 0.7128, 'score_p90': 0.7445, ...}}

$ python -m ops.deploy seuil faible_confiance 0.75 --auteur eq-ia --motif "P10 observe a 0,71 : capturer ce qui est sous la mediane"

$ # trafic suivant → captures automatiques
3 cas en attente de relecture
fa068bd0 confiance=0.7128 pseudonymise: {'MONTANT': 1, 'RAISON SOCIALE': 1}

$ # versement après relecture
verse dans le jeu d'eval : pfa068bd0 | clauses: [résiliation, durée, confidentialité] | seuil: 0.5

$ # le gate rejoue le cas, sans configuration supplémentaire
contrats evalues : 13 | dont issus de production : ['pfa068bd0']
pfa068bd0 -> note 0.6667

$ # journal de pilotage
publication · canary · promotion_canary · ajustement_seuil
· capture_faible_confiance x3 · enrichissement_jeu_evaluation
```

Démo en salle avec le vrai modèle (MOCK=off, Ollama + proxy de dérive) :

```
make up
python -m ops.deploy publier v2.0.0 && python -m ops.deploy canary v2.0.0 --pourcentage 20
python -m ops.deploy surveiller --boucle --intervalle 10 &
make traffic MODE=derive-score DUREE=120
# le tableau de bord (:8501) montre la confiance v2 décrocher, puis le canary disparaître
```

Pilotage depuis le tableau de bord

Les trois boucles sont déclenchables par HTTP (`/api/pilotage/surveiller`, `/api/pilotage/promouvoir`, `/api/captures/<id>/valider`) **uniquement si DEMO=on** . Hors de ce mode, ces routes répondent 403 et le pilotage passe par `python -m ops.deploy ...` .

Ce n'est pas un détail de confort. Exposer un rollback ou une promotion sur une page non authentifiée n'est défendable que sur une instance de démonstration, servie en `MOCK` et sans donnée client. Sur une instance qui traite de vrais contrats, le frontend reste en lecture seule et seules les personnes ayant un accès au service peuvent agir sur la chaîne.

Réservees

- L'instance de démonstration relève le seuil `faible_confiance` à 0,78 au démarrage (la confiance tourne autour de 0,73 en `MOCK` , donc au seuil de production la boucle 3 ne capturerait jamais rien et resterait invisible). L'écart est posé par `ajuster_seuil` , donc tracé au journal comme n'importe quel ajustement — il se voit, il ne se cache pas.
- Les chiffres du transcript viennent du mode `MOCK` : la confiance y est constante (0,86 déclarée par le repli), donc la confiance globale observée (0,735) est un artefact du mode, pas une mesure du modèle. **Les seuils de dérive doivent être recalibrés sur une semaine de trafic réel** avant d'armer la surveillance automatique en production.
- Le routage canary est un compteur cyclique, sans session collante : un même juriste peut voir alterner v1 et v2 d'une analyse à l'autre. Acceptable tant que les analyses sont indépendantes ; à revoir si un historique de conversation est ajouté (la demande « chatbot » de la direction générale).
- La traduction et le chatbot évoqués en fin d'expression de besoin ne sont pas traités : ce sont deux produits distincts, pas une variante de version du modèle d'analyse, et les mêler à cette livraison en ferait une v2 impossible à évaluer avec un jeu de contrats annotés.